

Data Structures (ETCCDS202)

Unit-2 Assignment: Sorting & Complexity Analysis

Program:	B.Tech (AI and ML)
Course Title:	Data Structures
Course Code:	ETCCDS202
Student Name:	Taruna Tewatia
Roll Number:	2501730115
Section:	A
Semester:	2
Batch:	2025-26
Submitted To:	Mrs. Neetu Chauhan
Submission Date:	

1. Introduction

Sorting is a fundamental operation in Data Structures used to arrange data in a specific order. Different sorting algorithms perform differently depending on the size and nature of input data. In this assignment, I implemented a toolkit in Python to understand algorithm paradigms and practical performance. The project includes implementations of **Insertion Sort**, **Merge Sort**, and **Quick Sort**.

2. Algorithms Implemented

2.1 Insertion Sort

Insertion Sort works by placing each element in its correct position in the sorted part of the array. It is efficient for small datasets or nearly sorted data.

- **Time Complexity:** Best Case $O(n)$, Worst Case $O(n^2)$
- **Stable:** Yes | **In-place:** Yes

2.2 Merge Sort

Merge Sort follows the **Divide & Conquer** approach. It recursively divides the array into halves, sorts them, and merges them. This ensures consistent performance regardless of input order.

- **Time Complexity:** $O(n \log n)$ for all cases
- **Stable:** Yes | **In-place:** No (requires extra space)

2.3 Quick Sort

Quick Sort selects a pivot and partitions the array around it. While typically faster in practice, its performance depends heavily on pivot selection.

- **Time Complexity:** Average $O(n \log n)$, Worst Case $O(n^2)$
- **Stable:** No | **In-place:** Yes

3. Experimental Setup

The algorithms were tested on three types of datasets (Random, Sorted, Reverse Sorted) with input sizes of 1000, 5000, and 10000. Execution time was measured in milliseconds (ms).

4. Results & Comparison

Type	Size	Insertion Sort (ms)	Merge Sort (ms)	Quick Sort (ms)
Random	1000	68.133	6.401	5.427

Type	Size	Insertion Sort (ms)	Merge Sort (ms)	Quick Sort (ms)
Sorted	1000	0.35	6.338	1.333
Reverse	1000	222.047	6.102	5.815
Random	5000	1359.743	31.897	14.396
Sorted	5000	1.374	17.017	25.534
Reverse	5000	1632.957	9.399	17.862
Random	10000	3336.415	53.963	52.983
Sorted	10000	3.608	40.079	49.165
Reverse	10000	5395.837	26.443	30.075

5. Analysis & Paradigms

Divide & Conquer: Binary Search and Merge Sort both follow this paradigm. In this experiment, Merge Sort consistently outperformed Insertion Sort on large datasets because it reduces the problem size by half at each step.

Practical Observation: Insertion Sort outperformed all others on *Sorted Data*, demonstrating its $O(n)$ best-case efficiency. However, it showed quadratic growth ($O(n^2)$) on random and reverse data, becoming extremely slow as size increased.

6. Conclusion

The choice of sorting algorithm depends on the data. Insertion Sort is ideal for small or nearly sorted lists. Merge Sort provides reliable, consistent performance for large datasets. Quick Sort is efficient but requires careful pivot selection to avoid worst-case scenarios.